

RUNBOOK ADMINISTRATEUR

Manuel administrateur et dossier technique

Reconstruction, exploitation, code, déploiement, alertes, sauvegardes, restauration et maintenance de Secure Local Cloud.

Edition 2026

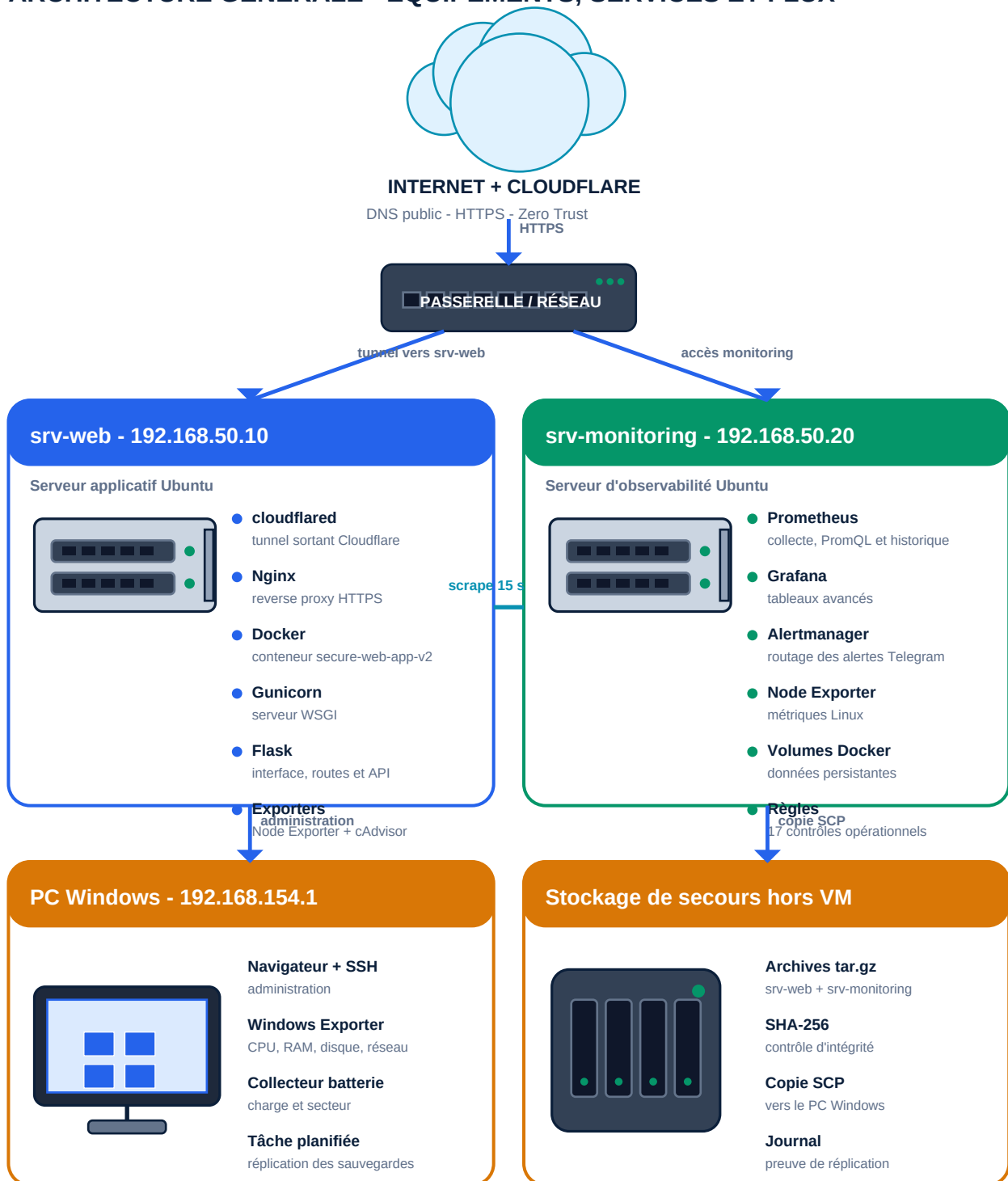
Document de référence - aucune valeur secrète n'est incluse.

Sommaire

Sommaire	2
1. Architecture complète des équipements	3
Architecture logique et flux	4
Réseaux utilisés	5
2. Inventaire détaillé par équipement	5
Ports et exposition	5
3. Montage initial	5
Préparer les machines	5
4. Installation de l'application	6
5. Installation du monitoring	6
6. Chemin HTTPS de bout en bout	6
7. Architecture du code	7
8. API multi-équipement	8
9. Alertes et Telegram	8
10. Sauvegardes et réplication	9
11. Procédure de restauration	9
12. Déploiement et retour arrière	10
13. Maintenance périodique	10
14. Publication GitHub	10
15. Checklist de reprise après plusieurs mois	10

1. Architecture complète des équipements

ARCHITECTURE GÉNÉRALE - ÉQUIPEMENTS, SERVICES ET FLUX

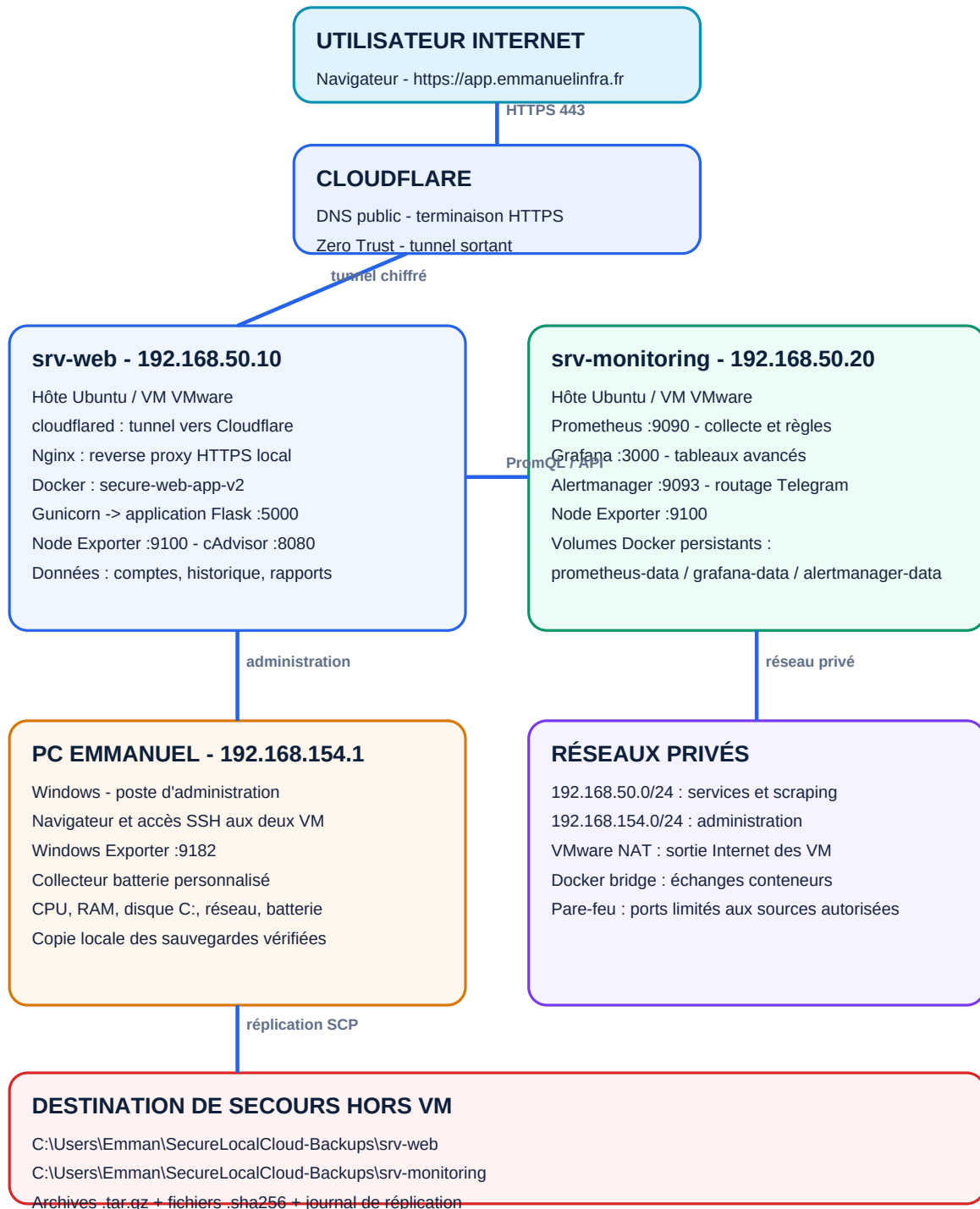


Réseau services : 192.168.50.0/24 | Réseau administration : 192.168.154.0/24

Chaque technologie est placée sur l'équipement où elle s'exécute réellement.

Architecture logique et flux

ARCHITECTURE PHYSIQUE ET EMPLACEMENT DES SERVICES



Réseaux utilisés

Réseau	Usage
192.168.50.0/24	Services et collecte entre srv-web et srv-monitoring.
192.168.154.0/24	Administration depuis Windows, SSH, réplication et Windows Exporter.
VMware NAT	Sortie Internet des VM pour mises à jour et Cloudflare Tunnel.
Docker bridge	Communication interne des conteneurs sur chaque hôte.

2. Inventaire détaillé par équipement

Équipement	Composants internes	Responsabilité
srv-web	Ubuntu, cloudflared, Nginx, Docker, Gunicorn, Flask, Node Exporter, cAdvisor	Publication HTTPS, application, API, conteneurs et métriques applicatives.
srv-monitoring	Ubuntu, Docker Compose, Prometheus, Grafana, Alertmanager, Node Exporter	Collecte, stockage temporel, visualisation, règles et notifications.
PC Emmanuel	Windows, VMware, navigateur, SSH, Windows Exporter, collecteur batterie, tâche de réplication	Administration, métriques Windows et copie hors VM.

Ports et exposition

Port	Service	Exposition attendue
443	Nginx / Cloudflare	Accès applicatif HTTPS.
5001	Gunicorn publié par Docker	127.0.0.1 uniquement.
9090	Prometheus	Interfaces privées autorisées.
3000	Grafana	Interfaces privées ou tunnel contrôlé.
9093	Alertmanager	Réseau de monitoring.
9100	Node Exporter	Prometheus uniquement.
8080	cAdvisor	Prometheus uniquement.
9182	Windows Exporter	srv-monitoring uniquement.

3. Montage initial

Préparer les machines

- Créer deux VM Ubuntu 24.04 avec trois interfaces selon le laboratoire.
- Attribuer les adresses privées fixes et conserver une route par défaut via l'interface NAT.
- Installer Docker Engine, Docker Compose, Nginx et les exporters nécessaires.
- Limiter les ports exportés aux réseaux d'administration et de services.

```
hostnamectl
cat /etc/os-release
ip -br address
ip route
timedatectl status
df -h
free -h
systemctl --failed --no-pager
```

4. Installation de l'application

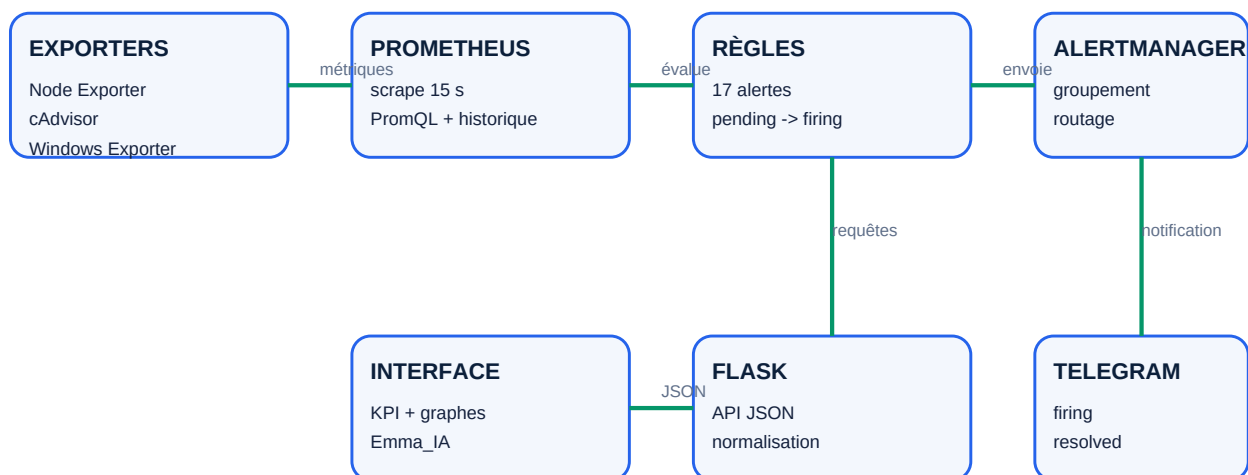
```
cd application
cp .env.example .env
chmod 600 .env
python3 -c 'import secrets; print(secrets.token_urlsafe(48))'
docker compose config --quiet
docker compose up -d --build
docker compose ps
docker compose logs --tail 100 web
```

Les secrets générés sont placés dans .env uniquement sur srv-web. Le fichier public .env.example ne contient que des noms de variables et des exemples neutres.

5. Installation du monitoring

```
cd ~/monitoring
docker compose config
docker compose up -d
docker compose ps
curl -fsS http://127.0.0.1:9090/-/ready
curl -fsS http://127.0.0.1:9093/-/ready
```

Prometheus collecte srv-web:9100, cAdvisor:8080, srv-monitoring:9100 et le PC Windows:9182. Les labels equipment, role et os permettent aux requêtes et alertes de rester génériques.



Principe essentiel : une métrique décrit un état chiffré ; une règle décide si cet état devient une alerte.

L'interface lit Prometheus mais ne remplace ni le stockage temporel ni le moteur d'alertes.

6. Chemin HTTPS de bout en bout

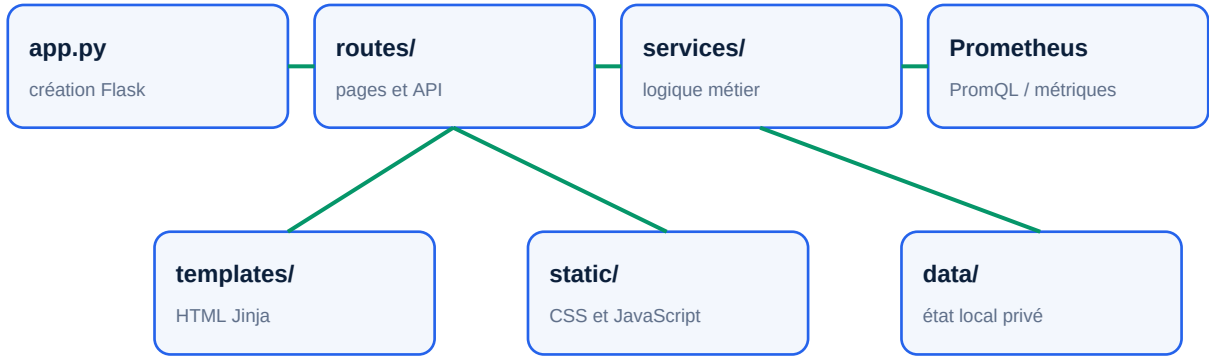
Cloudflare termine le HTTPS public et transmet la requête par un tunnel sortant cloudflared. Nginx reçoit l'origine sur srv-web et reverse-proxy vers Gunicorn exposé uniquement sur 127.0.0.1:5001. Gunicorn exécute Flask sur le port 5000 du conteneur.

Étape	Lieu	Fonction
1	Navigateur	Demande https://app.emmanuelinfra.fr .

Étape	Lieu	Fonction
2	Cloudflare	DNS, certificat public, filtrage et terminaison TLS.
3	cloudflared sur srv-web	Tunnel sortant ; aucun port entrant à ouvrir sur la box.
4	Nginx sur srv-web	Reverse proxy et en-têtes HTTP.
5	127.0.0.1:5001	Port Docker local vers Unicorn.
6	Conteneur :5000	Gunicorn distribue la requête à Flask.
7	Flask	Authentification, route, service, rendu HTML ou JSON.

```
sudo nginx -t
sudo systemctl reload nginx
systemctl status cloudflared --no-pager
curl -k -I https://127.0.0.1
curl -I https://app.emmanuelinfra.fr
```

7. Architecture du code



Flux principal : requête -> route -> service -> Prometheus -> réponse -> interface

Chemin	Responsabilité
application/app.py	Fabrique Flask, blueprints et initialisation.
application/config.py	Variables d'environnement et paramètres.
application/routes/	Pages, API, authentification, rapports et audit.
application/services/prometheus_service.py	PromQL Linux/Windows et normalisation.
application/services/assistant_engine.py	Routage général d'Emma_IA.
application/services/assistant_equipment.py	Réponses multi-équipement temps réel.
application/templates/	HTML Jinja.
application/static/	CSS, JavaScript et ressources visuelles.
application/data/	État local privé, non publié.
monitoring/	Prometheus, alertes, Alertmanager et Compose.
docs/	Documentation publique.
application/tests/	Tests de non-régression.

8. API multi-équipement

Route	Réponse
GET /api/equipment	Inventaire et disponibilité des trois équipements.
GET /api/equipment/<nom>/metrics	Valeurs instantanées, services, batterie et volumes.
GET /api/equipment/<nom>/history?hours=N	Séries temporelles CPU, RAM et disque.
GET /api/metrics	Compatibilité de l'ancien tableau de bord.
POST /api/assistant	Question Emma_IA après authentification.
GET /export/pdf	Rapport de supervision PDF authentifié.

9. Alertes et Telegram

Les règles sont dans `monitoring/alerts.yml`. Prometheus les évalue puis envoie les alertes firing à Alertmanager. Le routage Telegram est défini dans `alertmanager.yml`, mais le jeton et l'identifiant de discussion doivent être injectés hors Git.

État	Signification	Action
inactive	Condition fausse.	Aucune notification.
pending	Condition vraie mais durée for non atteinte.	Observer sans redémarrer précipitamment.
firing	Condition maintenue.	Alertmanager groupe et envoie Telegram.
resolved	Condition revenue à la normale.	Confirmer le service et documenter.

```
docker compose exec -T prometheus promtool check rules /etc/prometheus/alerts.yml
docker compose exec -T alertmanager amtool check-config /etc/alertmanager/alertmanager.yml
curl -fsS http://127.0.0.1:9090/api/v1/alerts | python3 -m json.tool
curl -fsS http://127.0.0.1:9093/api/v2/alerts | python3 -m json.tool
```

Ne jamais copier un fichier contenant un jeton réel vers GitHub, un PDF public, une archive partagée ou une conversation. En cas d'exposition, révoquer et recréer le jeton.

10. Sauvegardes et réplication



SERVEURS

COPIE HORS VM

Restauration : sélectionner l'archive -> vérifier SHA-256 -> extraire sur machine de test -> recréer volumes -> démarrer -> valider.

Une sauvegarde non restaurée au moins une fois reste une hypothèse, pas une garantie.

Étape	Destination
Sauvegarde srv-web	/var/backups/... puis /home/emmanuel/backup-export/srv-web/
Sauvegarde srv-monitoring	/var/backups/... puis /home/emmanuel/backup-export/srv-monitoring/
Empreinte	Fichier .sha256 adjacent à chaque archive .tar.gz.
Réplication Windows	C:\Users\Emman\SecureLocalCloud-Backups\srv-web et srv-monitoring
Journal	C:\Users\Emman\SecureLocalCloud-Backups\logs\replication.log
Tâche Windows	Secure Local Cloud - Replication, déclenchée automatiquement.

```
# Linux
systemctl list-timers --all | grep backup
sha256sum -c ./*.tar.gz.sha256

# Windows PowerShell
Get-ScheduledTaskInfo -TaskName 'Secure Local Cloud - Replication'
Get-ChildItem $env:USERPROFILE\SecureLocalCloud-Backups -Recurse -File
Get-Content $env:USERPROFILE\SecureLocalCloud-Backups\logs\replication.log -Tail 30
```

11. Procédure de restauration

- Choisir une archive et vérifier son SHA-256.
- Copier l'archive sur une machine de test, jamais directement en production.
- Extraire dans un dossier vide et inventorier configurations, données et volumes.
- Recréer les volumes Docker avant les conteneurs.
- Restaurer les permissions restrictives des secrets.
- Démarrer le monitoring puis l'application.
- Tester authentification, métriques, historiques, alertes, PDF et sauvegardes.
- Documenter le test avant d'autoriser une restauration de production.

12. Déploiement et retour arrière

```
# Avant modification
git switch -c feature/nom-du-changement
tar -czf ~/application-before-$(date -u +%Y%m%d-%H%M%S).tar.gz .

# Image de test
docker build -t application-web:change-test .
docker run --rm application-web:change-test python -m unittest discover -s tests -p 'test_*.py' -q

# Production
docker image tag application-web:latest application-web:before-change
docker image tag application-web:change-test application-web:latest
docker compose up -d --force-recreate --no-deps --no-build web
```

Conserver le tag before-change jusqu'à validation de la santé, des logs, de l'URL publique et d'un parcours fonctionnel.

13. Maintenance périodique

Fréquence	Actions
Chaque semaine	Vérifier timers, archives, SHA-256, réplication et journal Windows.
Chaque mois	Contrôler espace disque, volumes, cibles Prometheus et alertes Telegram.
Chaque trimestre	Mettre à jour dépendances et images après sauvegarde et tests.
Avant changement	Sauvegarde, branche Git, image de test et plan de retour arrière.
Avant migration	Restauration complète sur une machine de test.

14. Publication GitHub

```
git status --short
git diff --check
git add README.md application docs monitoring
git diff --cached --stat
git commit -m 'Documenter et finaliser le monitoring multi-équipement'
git push -u origin feature/multi-equipment-monitoring
```

Avant git add, vérifier qu'aucun .env, credential, archive, base, clé privée, jeton ou fichier de données n'est suivi.

15. Checklist de reprise après plusieurs mois

- Lire README.md puis docs/ARCHITECTURE.md.
- Vérifier les trois équipements et les routes réseau.
- Contrôler docker compose ps sur les deux serveurs.
- Vérifier les targets et règles Prometheus.
- Lire les derniers journaux Nginx, Flask, Prometheus et Alertmanager.
- Contrôler la dernière sauvegarde serveur et sa copie Windows.
- Créer une branche et une sauvegarde avant toute correction.